

Snack Time: Compaction Filters

Congratulations! You made it there! In the previous chapter, you made your LSM engine multi-version capable, and the users can use transaction APIs to interact with your storage engine. At the end of this week, we will implement some easy but important features of the storage engine. Welcome to Mini-LSM's week 3 snack time!

In this chapter, we will generalize our compaction garbage collection logic to become compaction filters.

For now, our compaction will simply retain the keys above the watermark and the latest version of the keys below the watermark. We can add some magic to the compaction process to help the user collect some unused data automatically as a background job.

Consider a case that the user uses Mini-LSM to store database tables. Each row in the table are prefixed with the table name. For example,

```
table1_key1 -> row
table1_key2 -> row
table1_key3 -> row
table2_key1 -> row
table2_key2 -> row
```

Now the user executes `DROP TABLE table1`. The engine will need to clean up all the data beginning with `table1`.

There are a lot of ways to achieve the goal. The user of Mini-LSM can scan all the keys beginning with `table1` and requests the engine to delete it. However, scanning a very large database might be slow, and it will generate the same number of delete tombstones as the existing keys. Therefore, scan-and-delete will not free up the space occupied by the dropped table – instead, it will add more data to the engine and the space can only be reclaimed when the tombstones reach the bottom level of the engine.

Or, they can create column families (we will talk about this in *rest of your life* chapter). They store each table in a column family, which is a standalone LSM state, and directly remove the SST files corresponding to the column family when the user drop the table.

In this course, we will implement the third approach: compaction filters. Compaction filters can be dynamically added to the engine at runtime. During the compaction, if a key matching the compaction filter is found, we can silently remove it in the background. Therefore, the user can attach a compaction filter of `prefix=table1` to the engine, and all these keys will be removed during compaction.

Task 1: Compaction Filter

In this task, you will need to modify:

```
src/compact.rs
```

You can iterate all compaction filters in `LsmStorageInner::compaction_filters`. If the first version of the key below watermark matches the compaction filter, simply remove it instead of keeping it in the SST file.

To run test cases,

```
cargo x copy-test --week 3 --day 7  
cargo x scheck
```

You can assume that the user will not get the keys within the prefix filter range. And, they will not scan the keys in the prefix range. Therefore, it is okay to return a wrong value when a user requests the keys in the prefix filter range (i.e., undefined behavior).

Your feedback is greatly appreciated. Welcome to join our [Discord Community](#).

Found an issue? Create an issue / pull request on github.com/skyzh/mini-lsm.

mini-lsm-book © 2022-2025 by Alex Chi Z is licensed under CC BY-NC-SA 4.0.